COMPTE RENDU DÉTAILLÉ — MINI-PROJET 2 KUBERNETES

Déploiement, persistance, sauvegarde et sécurisation de l'application TaskMaster

Projet réalisé sur un cluster K3s à trois nœuds — Namespace : task-app

Étudiant(e) : ..... Classe : ..... Date : .....

Remarque : compléter les captures et résultats de tests réels avant la remise.

1. Introduction et objectifs

Ce mini-projet consiste à déployer une application web de gestion de tâches, nommée TaskMaster, sur un cluster Kubernetes léger basé sur K3s. L'objectif ne se limite pas à démarrer des conteneurs : il s'agit d'organiser les composants, d'assurer leur découverte réseau, de conserver les données, de prévoir une procédure de sauvegarde et de réduire les risques liés aux accès et aux communications réseau.

La démarche suit plusieurs principes de Kubernetes : déclarer l'état souhaité dans des manifestes YAML, séparer les composants dans un namespace, exposer les applications au moyen de Services et d'un Ingress, externaliser la configuration, gérer les données persistantes avec des PersistentVolumeClaims, puis appliquer des contrôles de sécurité.

Les commandes et configurations présentées dans ce document sont des procédures de référence. Les éléments confirmés dans le contexte du projet sont notamment le cluster à trois nœuds, l'accès à l'application par Ingress et une réponse HTTP 200 obtenue avec un en-tête Host approprié. Les tests de politiques réseau et les tests de restauration doivent être documentés avec les sorties réellement observées avant la remise finale.

2. Architecture et rôle des composants

L'application suit une architecture trois tiers : un frontend Nginx qui sert l'interface web, une API Node.js/Express qui traite les requêtes applicatives et une base PostgreSQL qui stocke les données. Cette séparation permet d'isoler les responsabilités, de mettre à jour les composants indépendamment et de limiter les communications aux flux nécessaires.

Composant	Image / technologie	Port	Fonction
Frontend	sen5e1/task-frontend:1.2, Nginx	8080	Présentation et accès utilisateur
API	sen5e1/task-backend:1.2, Node.js/Express	3000	Logique applicative et accès aux données
Base de données	PostgreSQL 15 Alpine	5432	Stockage durable des tâches
Ingress	Traefik	HTTP/80 selon configuration	Routage externe vers le frontend

Le cluster comprend un nœud maître et deux workers. Les adresses relevées dans le projet sont 192.168.122.242 pour le master, 192.168.122.235 pour worker1 et 192.168.122.180 pour worker2. Ces adresses appartiennent à l'environnement de laboratoire et peuvent changer selon le réseau virtuel.

Le namespace task-app regroupe les ressources applicatives. Il fournit une frontière logique pour les quotas, les politiques de sécurité, les contrôles d'accès et l'administration des ressources. Les Services internes utilisent la découverte DNS Kubernetes : frontend vers api-svc:3000, puis API vers db:5432.

3. Déploiement et configuration

3.1 Namespace et limites de ressources

Le namespace task-app isole les ressources du projet des autres applications du cluster. Un ResourceQuota peut plafonner les ressources cumulées du namespace (CPU, mémoire, nombre de Pods ou de PVC), tandis qu'un LimitRange peut définir des demandes et limites par défaut pour les conteneurs. Ces mécanismes évitent qu'un seul projet consomme toutes les ressources disponibles.

```
kubectl apply -f k8s/00-namespace.yaml
kubectl apply -f k8s/01-resource-quota.yaml
kubectl apply -f k8s/02-limit-range.yaml
kubectl get resourcequota,limitrange -n task-app
```

3.2 Configuration et secrets

La ConfigMap task-app-config contient les paramètres non confidentiels : nom d'hôte PostgreSQL, port, nom de base, port de l'API et URL interne de l'API. La ConfigMap sépare la configuration de l'image et facilite les changements d'environnement sans reconstruire les images.

Le Secret db-credentials contient les identifiants de la base. Les valeurs réelles ne doivent pas être commitées dans Git. Le fichier 04-secret.example.yaml doit servir de modèle uniquement ; le Secret réel est créé localement ou par un mécanisme sécurisé. Un Secret Kubernetes est encodé en base64 par défaut, ce qui ne constitue pas un chiffrement suffisant à lui seul : les droits RBAC et la

configuration du cluster restent importants.

```
kubectl create secret generic db-credentials -n task-app \\
--from-literal=POSTGRES_USER='<utilisateur>' \\
--from-literal=POSTGRES_PASSWORD='<mot-de-passe>'
kubectl get configmap,secret -n task-app
```

3.3 Comptes de service

Les ServiceAccounts api-sa, frontend-sa et db-sa donnent une identité Kubernetes distincte aux workloads. Le compte dev-viewer est prévu pour une personne ou un outil ayant besoin d'inspecter certaines ressources. Lorsque l'application n'a pas besoin d'interroger l'API Kubernetes, automountServiceAccountToken: false réduit le risque qu'un jeton d'accès soit disponible inutilement dans le conteneur.

3.4 Déploiement de PostgreSQL

La base est déployée au moyen d'un StatefulSet nommé db. Contrairement à un Deployment généralement utilisé pour les composants stateless, un StatefulSet fournit des identités stables aux Pods et s'associe naturellement à des volumes persistants. Le PVC data-db-0 demande un volume à la StorageClass local-path.

Le Service headless db permet à Kubernetes de fournir une identité DNS stable au Pod de base de données. Le volume persistant conserve les fichiers de PostgreSQL au-delà de la suppression et de la recréation du Pod, tant que le volume et les données sous-jacentes ne sont pas supprimés. Il faut donc vérifier l'état du PVC et du Pod plutôt que de supposer que la persistance fonctionne.

```
kubectl apply -f k8s/06-postgres-service.yaml
kubectl apply -f k8s/07-postgres-statefulset.yaml
kubectl get pods,pvc -n task-app
kubectl describe pvc data-db-0 -n task-app
```

3.5 Déploiement de l'API et du frontend

L'API Node.js/Express écoute sur le port 3000 et communique avec PostgreSQL à travers le nom DNS db. Le frontend Nginx écoute sur le port 8080 et est exposé par le Service frontend-svc. Le Service api-svc fournit une adresse stable aux appels internes de l'application, même si le Pod API est remplacé.

Une difficulté rencontrée concernait le démarrage du frontend : le script d'entrée de l'image tentait de modifier un fichier de configuration monté en lecture seule. Une solution de contournement utilisée dans le projet consiste à lancer directement Nginx avec la commande nginx -g 'daemon off;'. Cette solution doit être testée avec les chemins de fichiers temporaires et de PID si readOnlyRootFilesystem est activé.

```
kubectl apply -f k8s/08-api.yaml
kubectl rollout status deployment/api -n task-app
kubectl apply -f k8s/09-frontend.yaml
kubectl rollout status deployment/frontend -n task-app
kubectl get deploy,svc,endpoints -n task-app
```

3.6 Ingress et accès externe

Traefik reçoit les requêtes HTTP et les dirige selon le nom d'hôte app.k3s.local vers le Service frontend. Le nom doit être résolu par le poste client vers une adresse IP joignable du cluster, ou être transmis explicitement dans l'en-tête Host lors d'un test curl.

```
curl -i -H 'Host: app.k3s.local' http://192.168.122.180
kubectl get ingress -n task-app
kubectl describe ingress -n task-app
```

Le test réalisé dans le projet a renvoyé HTTP 200 avec l'HTML de TaskMaster. Ce résultat confirme l'accès HTTP au frontend par ce chemin de routage ; il ne suffit pas, à lui seul, à prouver que toutes les opérations métier de l'API et de la base fonctionnent.

4. Sauvegarde et restauration PostgreSQL

4.1 Objectif et fonctionnement

Un volume persistant protège contre la perte liée à la simple recréation d'un Pod, mais ne protège pas contre une suppression accidentelle, une corruption ou la perte du nœud qui héberge les données. Le projet prévoit donc un PVC de sauvegarde backup-pvc et un CronJob db-backup qui exécute pg_dump et stocke un fichier de sauvegarde.

Le CronJob est adapté à une tâche périodique : Kubernetes crée un Job selon la planification, puis un Pod exécute la commande de sauvegarde. La sauvegarde doit utiliser les identifiants fournis par le Secret, joindre le Service db sur le port 5432 et écrire dans le volume de sauvegarde. Il faut vérifier les journaux et la présence d'un fichier non vide ; l'existence du CronJob seule ne prouve pas qu'une sauvegarde a réussi.

```
kubectl apply -f k8s/11-backup-pvc.yaml
kubectl apply -f k8s/12-backup-cronjob.yaml
kubectl get cronjob,jobs,pvc -n task-app
kubectl logs job/<nom-du-job> -n task-app
```

4.2 Déclenchement manuel et contrôle

```
kubectl create job --from=cronjob/db-backup db-backup-manual -n task-app
kubectl get pods -n task-app
kubectl logs job/db-backup-manual -n task-app
```

Pour le compte rendu, joindre la sortie de get jobs, les journaux du Job et une vérification du fichier de sauvegarde dans le volume. Éviter d'afficher les mots de passe dans les captures d'écran.

4.3 Restauration et limites

La restauration doit être testée sur une base cible adaptée, de préférence un environnement de test, avant toute intervention sur les données de production. Avec une sauvegarde SQL en texte, psql peut rejouer le fichier. L'option ON_ERROR_STOP permet d'arrêter l'exécution en cas d'erreur SQL, afin d'éviter de considérer comme réussie une restauration partielle.

```
# Exemple de principe à adapter au nom réel du fichier et au conteneur
psql -v ON_ERROR_STOP=1 -h db -U <utilisateur> -d <base_cible> -f /backup/backup.sql
```

Le stockage local-path est généralement lié au stockage local d'un nœud. Il convient à un laboratoire, mais n'offre pas à lui seul une redondance inter-nœuds. Pour une meilleure résilience, les sauvegardes doivent être copiées hors du cluster ou vers un stockage indépendant, et des restaurations périodiques doivent être réalisées.

5. Sécurité : durcissement des Pods

Le durcissement vise à réduire les privilèges disponibles en cas de compromission d'un conteneur. Pour l'API et le frontend, les paramètres recommandés incluent runAsNonRoot: true, allowPrivilegeEscalation: false, capabilities.drop: ["ALL"] et seccompProfile.type: RuntimeDefault.

```
securityContext:
runAsNonRoot: true
allowPrivilegeEscalation: false
capabilities:
drop: ["ALL"]
seccompProfile:
type: RuntimeDefault
```

readOnlyRootFilesystem: true peut être activé lorsque l'image le permet. Les chemins qui doivent recevoir des fichiers temporaires doivent alors être montés depuis des volumes emptyDir. Pour Nginx, il faut examiner les emplacements du PID, du cache et des fichiers temporaires. Si l'application exige d'écrire dans le système de fichiers racine, cette option doit être adaptée plutôt qu'activée sans test.

PostgreSQL nécessite un accès en écriture à son répertoire de données persistant. Les contraintes de sécurité doivent respecter l'utilisateur et les permissions attendus par l'image PostgreSQL ; une exception doit être documentée et limitée aux besoins réels du service.

5.1 Pod Security Standards

Les Pod Security Standards proposent notamment les niveaux privileged, baseline et restricted. Le niveau baseline bloque certains comportements risqués ; restricted applique des exigences plus strictes. L'application d'un niveau plus strict peut révéler des incompatibilités d'images ou de permissions. Il est donc utile de commencer par vérifier les Pods existants, puis d'appliquer le niveau choisi et de corriger les avertissements avant d'enregistrer les résultats.

```
kubectl label namespace task-app pod-security.kubernetes.io/enforce=baseline --overwrite
kubectl get pods -n task-app
kubectl describe pod <nom-du-pod> -n task-app
```

Ne pas affirmer que le niveau restricted est validé sans avoir observé la création effective des Pods et les éventuels refus d'admission. Conserver les messages d'avertissement ou d'erreur comme éléments de preuve.

6. RBAC et moindre privilège

RBAC (Role-Based Access Control) contrôle les opérations qu'une identité peut effectuer sur les ressources Kubernetes. Le rôle dev-viewer-role est prévu pour un accès de lecture limité dans le namespace task-app : consulter les Pods, suivre leurs journaux et lire les Services. Le RoleBinding associe ce rôle au ServiceAccount dev-viewer.

Le principe du moindre privilège consiste à accorder uniquement les permissions nécessaires. Un compte de lecture ne devrait pas pouvoir supprimer des Pods, modifier des Services ou lire les Secrets. Les règles doivent préciser les apiGroups, resources et verbs autorisés.

```
kubectl auth can-i list pods -n task-app --as=system:serviceaccount:task-app:dev-viewer
kubectl auth can-i get pods/log -n task-app --as=system:serviceaccount:task-app:dev-viewer
kubectl auth can-i get secrets -n task-app --as=system:serviceaccount:task-app:dev-viewer
kubectl auth can-i delete pods -n task-app --as=system:serviceaccount:task-app:dev-viewer
```

Les deux premières commandes devraient répondre yes si le rôle et le binding sont correctement appliqués ; les accès aux Secrets et la suppression des Pods devraient répondre no en l'absence d'autres autorisations. La simulation avec --as peut exiger des droits d'impersonation pour l'utilisateur qui exécute kubectl. Si le test échoue pour cette raison, cela ne prouve pas automatiquement que le rôle est incorrect.

7. NetworkPolicies et segmentation réseau

Les NetworkPolicies permettent de limiter les communications réseau des Pods sélectionnés. Une stratégie de type default-deny ingress dans le namespace bloque les connexions entrantes non explicitement autorisées vers les Pods concernés. Des politiques supplémentaires ouvrent uniquement les flux attendus : Traefik vers frontend, frontend vers API, API vers PostgreSQL et Job de sauvegarde vers PostgreSQL.

Source	Destination	Port	Résultat attendu
Traefik (kube-system)	frontend	port frontend	Autorisé
Frontend	api-svc	TCP 3000	Autorisé
API	db	TCP 5432	Autorisé
Job de sauvegarde	db	TCP 5432	Autorisé
Frontend	db	TCP 5432	Refusé
Pod non autorisé	API / DB	Selon service	Refusé

La sélection par labels est essentielle : les labels de tier ou d'application doivent être cohérents entre les Deployments et les NetworkPolicies. Pour autoriser Traefik, une namespaceSelector peut cibler le namespace kube-system avec le label Kubernetes standard kubernetes.io/metadata.name=kube-system. Le flux réel doit être testé, car une erreur de label peut bloquer l'application.

Une NetworkPolicy qui ne définit que ingress contrôle les connexions entrantes, mais ne restreint pas automatiquement les connexions sortantes. Si le scénario exige une restriction des sorties, il faut aussi définir des règles egress et autoriser explicitement DNS ainsi que les destinations nécessaires.

7.1 Tests à documenter

```
kubectl get networkpolicy -n task-app
kubectl describe networkpolicy -n task-app
# Depuis un Pod de test autorisé : tester une connexion TCP vers db:5432
# Depuis un Pod sans le label autorisé : répéter le test et vérifier le refus
```

Les tests de connexion doivent être effectués depuis des Pods réellement présents dans le cluster. Une commande curl HTTP n'est pas un test approprié pour PostgreSQL ; utiliser un client TCP ou PostgreSQL adapté. Ajouter les sorties réelles et noter clairement les tests réussis, échoués ou non exécutés.

8. Vérification globale et résultats

8.1 Contrôles de santé

```
kubectl get nodes -o wide
kubectl get all -n task-app
kubectl get pvc -n task-app
kubectl get ingress -n task-app
kubectl get events -n task-app --sort-by=.lastTimestamp
```

Les nœuds doivent être Ready, les Pods attendus Running ou Completed selon leur rôle, et les PVC Bound. Si un Pod est en CrashLoopBackOff, consulter ses journaux et la description du Pod avant de modifier les manifestes. Les événements Kubernetes aident à identifier les erreurs de montage de volumes, de planification, d'image ou de permissions.

8.2 Éléments de preuve à joindre

Pour rendre le compte rendu vérifiable, ajouter des captures d'écran datées des sorties kubectl get nodes -o wide, kubectl get pods -n task-app, kubectl get pvc -n task-app, kubectl get ingress -n task-app, de la réponse HTTP 200, des journaux d'une sauvegarde réussie, des résultats RBAC et des tests NetworkPolicy. Ne pas inclure de mots de passe ni de valeurs confidentielles.

Une vérification fonctionnelle complète doit inclure plus que l'ouverture de la page : créer une tâche, vérifier qu'elle apparaît dans l'interface, confirmer sa présence en base, supprimer le Pod PostgreSQL sans supprimer le PVC, puis vérifier que les données sont toujours disponibles. Tester séparément la restauration à partir d'une sauvegarde permet de valider la procédure de reprise.

9. Conclusion

Ce mini-projet met en pratique plusieurs fonctionnalités essentielles de Kubernetes : orchestration de composants, découverte de services, exposition par Ingress, configuration externalisée, stockage persistant, sauvegarde planifiée et contrôle des accès. Il montre également qu'un déploiement fonctionnel et un déploiement sécurisé ne sont pas équivalents : les privilèges des conteneurs, les permissions RBAC, les communications réseau et la stratégie de sauvegarde doivent être validés séparément.

Les résultats déjà observés confirment l'accès HTTP au frontend par Traefik et le routage associé au nom d'hôte app.k3s.local. Pour conclure complètement le projet, il reste important d'insérer les preuves réelles de persistance, de sauvegarde/restauration, de contrôle RBAC et de segmentation réseau, puis de documenter les éventuels écarts et corrections.